DESIGN SPECIFICATION / REVISION 3

Performance and trace harness

Measurement methodology, workload format, run modes and acceptance rules for svg-dom-graph

> **DECISION IN ONE SENTENCE**
>
> Build one harness with three measurement layers (pure Rust, WASM/DOM, real browser) and four run modes (correctness, performance, recovery, endurance), driven by a versioned workload manifest. It records raw observations and never lets a timing count unless the same run's checkpoint matches the reference model.

Prepared: 5 November 2026. **Target:** main at `a49c0bb`, crate 0.2.47. **Origin:** audit deliverable 1 / P0, and a review of revision 1 that found the timing, statistical, workload-format and layering rules missing. **Status:** Phases 0, 1 and 2 complete (findings in `perf/README.md`, raw evidence in `perf/evidence/phase0/`, schemas in `perf/schemas/`). Revision 3 applies the review of revision 2. Phase 1 is described in section 5.1 and Phase 2 in section 5.2. Phase 3 is conditional on section 3.

> **WHAT THIS HARNESS CAN AND CANNOT CLAIM**
>
> It establishes a documented, repeatable operating envelope for named browsers, devices and workloads. It does not by itself prove production readiness, and a flat memory trend does not prove that every resource was released. A rising trend is a prompt to investigate, not a verdict of a leak.

1 Scope

- In scope: the measurement rules, the manifest and result schemas, the checkpoint contract, the runners, the statistical comparison and the CI profiles.
- Out of scope: the stress demos themselves, any new library API, line coverage, and Firefox or Safari timing in the first release.
- Kept outside the published library's runtime API. An instrumentation hook is added to the library only if a Phase 0 finding shows it is the sole way to observe something, and then as its own change.
- Reference workloads for validating the harness, before any large simulation: **W1** scene construction, **W2** connector rerouting during node movement, **W3** repeated scene creation and destruction (build on a fresh `<svg>`, drop the last `Scene` handle, remove the element).

2 Three measurement layers

A slow drag has several possible causes: connector maths, WASM-to-JavaScript calls, SVG attribute writes, browser layout and paint, or accessibility relationship updates. One latency number cannot separate them, so every workload is measured per layer, and results are never merged across layers.

Layer	What it times	Runner	Tells you where to optimise
A Rust	DOM-free work: connector geometry, label placement, layout, formatting, the demos' own algorithms.	Native binary in `perf-harness`, same statistics code as the other layers.	The library or application algorithm.
B WASM/DOM	Public `Scene` calls and their synchronous SVG mutations. Returns when the call returns. Not a frame time.	A harness wasm module loaded in a page; timed in page with `performance.now()`. Chrome through CDP first; Firefox later through the existing WebDriver setup.	svg-dom-graph, or svg-dom beneath it.
C Browser	Input to visual update, frame behaviour, accessibility tree, memory and listener counts.	CDP runner against the built gallery and the fixture, using real `Input.dispatchMouseEvent` input.	The browser, the page, or the application.

2.1 Layers are never subtracted

Layer A runs native machine code and Layer B runs WebAssembly, so the same algorithm costs differently in each. The difference of an A and a B figure is not the cost of the DOM or of the boundary, and no report presents it as one. A comparison may be shown as exploratory. Isolating DOM overhead would need matched workloads measured in one execution environment, and is not part of this design.

A Layer B figure is called a *synchronous API time* in every report. The words "frame", "render" and "paint" are reserved for Layer C.

3 Measurement boundaries

Each metric has one fixed start and stop. A runner may not report a metric under any other definition.

Metric	Starts	Stops	Layer and notes
Construction	Immediately before the first scene API call	The last synchronous scene API call returns	B. Excludes wasm instantiation, which is a separate cold-start metric.
Mutation	Immediately before the targeted API call (for example `Scene::move_node`)	The call returns	B. Includes Rust work, boundary crossings, SVG writes and connector updates, which cannot be separated at this layer.
Rust-only	Before the DOM-free function	It returns	A. Reported on its own. Never subtracted from a B figure (see 2.1).
Presentation	A user-timing mark written immediately before the mutation, in the browser's own clock	The frame that the correlation rule in 3.2 attributes to that mutation	C, Chrome only. Where correlation fails, the sample is `unavailable`. Elsewhere the fallback is the second `requestAnimationFrame` callback followed by a `MessageChannel` task, reported as *presentation proxy*, which shows a frame began and not that pixels were painted.
Interaction	The first call of a capture-phase listener for the input event, stamped with `performance.now()` in the page	The presentation boundary of the same sample	C. Both ends are in the browser's clock. The CDP command's round-trip time is recorded as a separate metric and is never used as the start. Event Timing entries are kept alongside, noting their 8 ms rounding.
Teardown	Immediately before releasing the scene	Synchronous cleanup returns	B. Resource counts are read after a settle step (section 6), not inside this interval.
Recovery	Immediately before fault injection	The documented consistent state is reached and its checkpoint matches	Recovery mode only. Reported as its own metric, never mixed into latency series.
Cold start	Navigation begins	Gallery ready	C. Separate from steady state.

3.2 Correlating a mutation with a frame

The next `Commit` or `Paint` event after a mutation is not necessarily the one it caused, because unrelated browser work can produce an earlier event. The runner therefore does the following.

1. Records a CDP trace with the `blink.user_timing` and `devtools.timeline` categories, so the harness's marks and the frame lifecycle events share one clock.
2. Writes a uniquely numbered mark before each mutation, and attributes to it the first frame that begins after the mark and whose lifecycle includes a style or layout step.
3. Marks the sample `unavailable` when no such frame is found within a bound, when two candidates cannot be told apart, or when the trace has a gap.

This rule is a design, not a result. Phase 3 begins with a spike that shows it attributes frames correctly on a known workload before any Layer C figure is relied on.

3.1 Timer resolution and batching

- `performance.now()` is coarsened by browsers. Phase 0 measured 100 µs in headless Chrome 157 and 5 µs when the page is served with COOP and COEP headers. The harness serves its own pages with those headers and records the measured resolution in every result.

- When an operation takes less than 50 times the measured resolution, it is timed in a **batch** of *k* operations and reported as batch throughput, with *k* recorded. A batch figure is never labelled as individual latency.
- Timing code runs outside the measured interval. The harness does not call `console`, allocate result objects or touch the DOM inside it.

4 Run modes

Modes are separate executions. Fault injection never runs in a performance mode, so a latency figure never contains an injected error or its recovery.

Mode	Purpose	Rules
Correctness	Verify outcomes and invariants against the reference model.	Checkpoint after every batch. Dangling endpoints, duplicate ids and model/DOM mismatch fail the run. No timing is kept.
Performance	Measure successful operations.	No faults. Timed regions contain no checks. The checkpoint is taken after the timed region and compared with the reference; a mismatch marks every figure from that run `invalid_incorrect` and excludes it from comparison.
Recovery	Exercise deliberate failures and verify eventual consistency.	Each fault point declares its documented commit contract (section 7). Recovery time is its own metric.
Endurance	Repeated construct / use / destroy cycles.	Resource counts and sampled memory over many cycles; trend reported with its mechanism (section 6).

5 Workload manifest and determinism

A manifest is a versioned TOML file. The same manifest must produce the same logical operation stream on every run, regardless of timing, machine or browser.

```
schema_version = 1
workload       = "connector_rerouting"     # W2
seed           = 12345
prng           = "splitmix64"              # fixed algorithm, implemented in the harness

[graph]
nodes = 1000
edges = 3000
maximum_degree = 16

[execution]
modes                  = ["correctness", "performance"]
warmup_iterations      = 100
measurement_iterations = 1000
repetitions            = 5               # independent runs, each a fresh browser tab

[measurement]
capture_dom_counts   = true
capture_heap         = false
capture_frame_timing = true
checkpoint_every     = 100               # correctness mode only

[limits]
dom_nodes = 50000
wall_ms   = 120000
```

Values here are illustrative, not recommended thresholds.

- **Own PRNG.** The generator is a small fixed algorithm in the harness, not an external crate, so a dependency upgrade cannot change a seed's meaning.
- **Operation-stream hash.** Each operation is encoded canonically and hashed with SHA-256 in order. The encoding begins with a version byte. Every value carries a one-byte type tag. A string is a little-endian `u32` byte length followed by UTF-8. An integer is little-endian at a fixed width. A float is its IEEE-754 bit pattern, little-endian, with `-0.0` written as `+0.0`. NaN and infinity are rejected when a manifest is read, as the library rejects them. The hash is stored in the manifest's result, and a run fails if the executed stream hashes differently from the generated one. That proves the benchmark exercised the intended workload.
- **Reference model.** The generator also produces the expected checkpoint sequence from a DOM-free model, so correctness is checked against an independent computation.
- **Limits.** Exceeding one stops the run with `refused` or `timed_out`. Graceful refusal is never reported as a pass.

5.1 The workload engine (Phase 1)

The engine is the `perf-harness` crate. It reads a manifest, writes the operation stream, and drives a DOM-free reference model through it. It is described in full, with its evidence, in `perf/README.md`. This section records what the design now commits to.

Operation	Maps to
`create_scene`, `drop_scene`, `remove_host`	`Scene::new` on a fresh `<svg>`; dropping the last handle, which leaves the DOM; removing the `<svg>`.
`add_node`	`Scene::add_node`, a plain node at a size the caller gives, so the model knows the box exactly.
`add_edge`	`Scene::add_edge_with` with a straight connector. The library's default is an elbow, so a runner must ask for a straight one.
`move_node`	`Scene::move_node`.
`phase`, `checkpoint`	Markers. They change no scene, and a performance run skips the checkpoints.

- **Two digests.** A report carries `sha256` over every operation, which a correctness run executes, and `mutations_sha256` over the operations that change the scene, which a performance run executes. A result states which one it ran (`operation_stream_scope`). Neither depends on the repetition count, the limits or the capture options, and the mutations digest does not depend on where the checkpoints fall.
- **W1 and W3 differ in the graph.** W1 builds the same graph on every cycle and measures construction. W3 builds a different one each cycle, so that freed capacity has to be reused. W2 builds once and then moves nodes, one move in four aimed at the node of highest degree.
- **What the model predicts.** Topology, content, interaction, node boxes, connector ends and paint order are all predicted, and Phase 2 confirmed them against Chrome 157 for plain nodes and straight connectors. A straight connector's ends are where the ray between the two centres crosses each boundary, and where the two ends are one point the library draws no path and the model records none. Paint order is creation order, and a move does not raise a node. Ids and the accessibility tree are not predicted and are not compared.
- **Where it departs from section 5.** Coordinates are integers, so the float rule of the encoding is implemented and tested but no operation uses it yet. A seed written in TOML stops at $2^{63} - 1$.
- **Determinism.** The same manifest gives the same stream on native code and on 32-bit wasm, and an independent JavaScript generator gives the same counts and digests for 30 manifests. It has not yet been observed on a second physical machine, which the first run of the CI job does.

5.2 The runners (Phase 2)

Layer A is native and Layer B is a wasm module driving a real scene in Chrome. Both are described in full, with their evidence, in `perf/README.md`. This section records what the design now commits to.

- **Layers are reported apart.** Layer A times the library's own `boundary_point` for each connector at the moved node. Layer B times synchronous `Scene` calls. Neither is subtracted from the other, and neither is a frame time.
- **Where the clock is read.** `performance.now()` is read only where one kind of operation gives way to another, never inside an operation. Checks, resource counts and the model run between segments. Moves are timed in batches sized from the warm-up so that one observation spans 50 timer steps. A timing under 50 steps that could not be batched is flagged `resolution_limited` in the result.
- **What is observed.** The runner reads node boxes, edges, connector ends and paint order back out of the document, and the view from the scene, so a checkpoint is compared with what exists and not with what the runner meant. The comparison names the element, the coordinate, both values and the distance.
- **A performance run still proves it was right.** It skips the checkpoints along the way and executes the last two outside its timing. Its digest is the mutations digest, and a run that did not execute exactly that stream is invalid.

- **The checkpoint check is itself checked.** A self-test corrupts what was observed in six ways. Each must make the run invalid and name the right section, and a clean run must pass.
- **The counting allocator is verified before it is used.** It must add exactly the size of each retained block, return exactly on release, and count closures leaked with `Closure::forget`, including the library's own. The orchestrator checks it once per run and withholds every allocator figure if it fails.
- **The harness's own memory must not be measured as the library's.** Everything a run records is allocated at full size before the first reading, and a cycle that ran a checkpoint is left out of the endurance series. Without that, the harness's vectors doubling looked like a leak.
- **Limits.** The vocabulary registers no listeners, so the listener gate holds at zero and proves little until a workload makes nodes draggable. Firefox was not run, and recovery mode is Phase 4.

6 Statistical procedure

- **Phases.** Cold start is measured once per repetition and reported separately. A warm-up phase of fixed iterations is discarded from steady-state statistics but its raw values are kept. The measured phase has a fixed iteration count, never a fixed "until stable" rule.
- **Repetitions.** Each repetition is a separate experimental run in its own Chrome process with its own profile directory. They share the machine, its load and its thermal state, so they are not fully independent environments, and reports say so. For a baseline comparison, runs of baseline and candidate alternate (ABAB) so that drift is shared. Variation between repetitions is reported separately from variation within one.
- **Raw data.** Every observation is written to a samples file next to the result. Nothing is dropped. Outliers are flagged (Tukey fences, 1.5 × IQR) and counted, never removed.
- **Percentiles.** p50 needs at least 30 samples, p95 at least 200, p99 at least 1000. Below that, the report gives only min, median and max and says why. A percentile is given with a 95% interval from a hierarchical bootstrap: resample the repetitions, then resample the observations within each chosen one. The sample count sits beside it. These minimums set when a figure is shown and do not guarantee a reliable interval.
- **Comparison.** The unit is the repetition. The statistic is the ratio of candidate to baseline repetition medians, with a bootstrap interval across repetitions. With a ratio tolerance δ the verdict is `regression` (interval wholly above 1 + δ), `improvement` (wholly below 1 / (1 + δ)), `no_material_change` (wholly inside both bounds) or `inconclusive`. Inconclusive is never reported as a pass.
- **Few repetitions.** With the three repetitions of `perf-smoke`, results are labelled diagnostic and no verdict is issued.
- **Same environment.** A comparison is made only between runs whose keys match: OS, CPU model, runner class, the *full* browser version, and the harness's measurement protocol version. Otherwise the report says "not comparable".

6.1 Resources and memory

Kind	How measured	Cross-browser?
Deterministic counts	SVG element count by kind through the DOM; a listener tally; outstanding callbacks and scene-owned objects where the harness page can observe them. The tally counts effective registrations: it keys each by target, type, listener and capture flag, ignores a duplicate as the platform does, and handles `once`, `AbortSignal` and removal. It is cross-checked against Chrome's own count. It is installed before the wasm module loads.	Yes. These are the gate-eligible figures. Scene-owned listeners are released when the last `Scene` handle drops, so the gate is the count after a drop equalling the count before the build (see `perf/README.md`, finding 7).
Wasm capacity	The byte length of the module's linear memory, as a `capacity` metric: a high-water mark. It grows in pages and never shrinks, so it shows what was ever needed and not what is live. Reported as a diagnostic, apart from heap and listener figures.	Yes, in Layers B and C.
Wasm live allocation	Live bytes and allocation count from a counting global allocator in `perf-harness-wasm`, which the library links into unchanged. The library deliberately leaks some closures (`Closure::forget`), and this is the figure in which that would show. Not available in the demo gallery's module.	Layer B only. A design to be proved in Phase 2.
Browser counters	Chrome `Performance.getMetrics` (`Nodes`, `JSEventListeners`, `JSHeapUsedSize`) after `HeapProfiler.collectGarbage`. Measurement mechanism recorded with every figure.	No. Chrome-only diagnostics. Never an acceptance criterion in other browsers.
Endurance trend	Counts and heap sampled at the end of each cycle after a settle step (forced GC and two frames). Trend is a least-squares slope with its interval.	Counts yes; heap Chrome only.

Phase 0 found no `on*` property handlers in the library. The strongest lifecycle evidence combines count assertions, the existing listener-teardown tests and repeated create / destroy.

6.2 Reading wasm memory

Linear-memory size is capacity, not live Rust memory. A constant size can conceal retained allocations, and growth does not establish a leak. A lifecycle test therefore does not expect the size to return to its starting value after teardown. After a warm-up it counts how often memory grew during the measured cycles, which should be zero once capacity is being reused, and it compares live bytes after teardown with live bytes before the build. Growth in capacity with flat live bytes is reported as an observation, never as a leak.

6.3 Teardown observation point

Forced garbage collection is a Chrome diagnostic. It is not required for correctness and does not guarantee that every object is reclaimed. In Phase 0, Chrome's listener count read 139 before a collection and 37 after. Counts are compared with a baseline captured at the equivalent lifecycle point, for example the same scene built once and not yet dropped. The library has no public teardown call, so a teardown here means dropping the last `Scene` handle and removing the host `<svg>`. Leaving a gallery panel is not a teardown, because panels are built once and only hidden.

7 Checkpoints and failure contracts

A checkpoint is a structured snapshot with separately comparable sections, defined in `perf/schemas/checkpoint.schema.json`. Full numeric values are kept. A mismatch report names the section, and for geometry gives the element, coordinate, expected value, actual value and absolute difference.

Section	Content	Comparison rule
Topology	Nodes and edges by logical key (the name the scene was given), with endpoints; duplicate and dangling findings.	Exact, with a hash for diagnosis. Compared by logical key, not by DOM id.
DOM identity	Set of rendered element ids; uniqueness.	Uniqueness is always checked. Equality across scenes is not, because Phase 0 showed the arrow marker id comes from a process-wide counter (`svg-dom-graph-arrow-N`).
Geometry	Node boxes, connector paths, label positions in scene units.	Compared directly within 0.01 units on the full values. Never hashed: rounding before hashing can split two values that are well within tolerance (10.0049 and 10.0051).
Content	Data values, labels and text as displayed.	Exact.
Interaction	Selection, focused element, view transform.	The manifest states one of three things: the view is expected unchanged, the view is expected to end at a stated transform (a deterministic pan workload verifies it), or it is ignored.
Accessibility	Roles, names, relationships (Chrome AX tree in Layer C; DOM attributes elsewhere).	Exact, compared separately from the rest.
Paint order	Logical keys in paint order, where z-order matters.	Order significant, compared by logical key and not by DOM id. Other child ordering is sorted away.

Hashes are stored for the exact sections only, to identify which one differs.

7.1 Recovery mode

A failure before a change commits is not the same as a failure after part of it has. For each fault point the manifest names one of two expected outcomes, taken from the library's documented behaviour:

- **Nothing committed:** the checkpoint equals the one taken before the operation.
- **Partly committed:** a documented set of sections differs (for example, the node moved but its connector did not follow), and a defined retry brings the checkpoint to the reference.

Fault points: node construction, DOM transform, connector update, label update, scheduled frame, teardown, plus mid-drag teardown and a removed host SVG. Phase 4 begins by tabulating each point's actual contract

from the code; any point without a documented contract is reported as a finding, not tested against a guess.

8 CI profiles and acceptance

Profile	Content	When
`correctness`	Layer A and B small workloads; checkpoints; deterministic gates.	Per commit, blocking.
`perf-smoke`	W1 to W3 at small size, 3 repetitions, Layers A to C.	Per commit, **reporting only**.
`gallery`	Every gallery panel loaded and revisited; steppable demos walked.	Nightly.
`endurance`	Hour-scale cycles with sampled counts and heap.	Weekly or manual, on a fixed runner.

8.1 Gates

- **Deterministic gates (blocking from the first release):** checkpoint correctness, DOM and listener counts at teardown returning to baseline, resource limits, malformed-manifest rejection, the operation-stream hash.
- **Statistical gates (reporting only until at least 10 baseline runs exist on one runner class):** section 6's verdicts. Thresholds are then set from the observed run-to-run interval, not guessed.

8.2 Results and history

One JSON result per run, and a separate comparison report per pair of run collections. A result holds no between-run statistic or interval, since one run has neither. It carries a complete copy of its manifest, so it stands alone, with the manifest hash and the operation-stream hash. A result has `schema_version`, environment (browser and version, OS, CPU, memory, rustc, wasm-pack, crate version, git SHA, dirty flag, timer resolution), manifest copy and hash, mode, status (`ok`, `invalid_incorrect`, `refused`, `timed_out`, `error`), the metrics of section 3 with sample counts and within-run spread, and a pointer to the raw samples file. Each metric has its own status, `measured`, `unavailable` or `measurement_error`, so a trace or heap sample that fails does not turn a correct workload into a failed one. The schemas are `perf/schemas/manifest.schema.json` and `result.schema.json`. They are strict: a fixed object rejects unknown keys, and a metric's status decides which fields it must or must not carry (a measured one needs its unit, mechanism, sample count and summary; an unavailable one needs a reason and no figures). Validation is a deterministic gate, and `perf/schemas/check.mjs` runs every valid example and every deliberately broken one, so a schema that becomes too permissive fails.

The comparison report (`comparison.schema.json`) names the baseline and candidate runs and commits, and records the baseline's promotion record, the compatibility key, the bootstrap settings, the ratio tolerance and the confidence level. For each metric it gives each side's repetition count, median and interval, between-run and within-run spread, the ratio with its interval, and the verdict. Baselines are committed under `perf/baselines/` only by an explicit promotion command, which records the commit, the workload hash, the environment and the comparison method. A baseline is never updated by a run, and never after a regression without that command. Historical runs are CI artifacts, not committed, and a comparison report lists the baseline and candidate SHAs.

9 Existing demos

The gallery adapter lives in the Layer C runner. It selects a panel through its menu entry, so it works for every panel without changes. A panel with Next, Previous or Restart controls, found by accessible name, is also walked (forward then back, restart), and one with a test clock is advanced deterministically. A panel without them is measured for load and revisit only. Per-panel overrides are allowed where a control name is ambiguous, and a missing control downgrades a panel instead of failing the sweep.

Limit found in Phase 0. The gallery builds each panel once and later only hides it, so selecting and leaving a panel is not a teardown. The adapter reports build cost, revisit cost and the counts after a visit, and says so. Teardown is measured only by W3 and by the demos that stop their own clocks on `pagehide` (N-Queens, Tarjan, springs).

10 Phases

Phase	Deliverable	Status and exit condition
0	Methodology, schemas, spike.	**Complete.** Schemas are in `perf/schemas/` and the four spike answers are in `perf/README.md`: panels are not torn down on leaving; ids are not stable across scenes; there are no `on*` handlers; timer resolution is 100 µs, or 5 µs when isolated. It also found that leaked closures are invisible to the JS heap, and one listener result that a registration tracker then explained: a scene's listeners belong to its `Scene` handle and are released when it drops, while the DOM stays.
1	Deterministic workload engine and reference model for W1 to W3.	**Complete.** The `perf-harness` crate, the workload-report schema and three reference manifests. The same seed gives identical operations, digests and checkpoints on native code and on 32-bit wasm, and an independent JavaScript generator agrees. Not yet seen on a second physical machine (section 5.1).
2	WASM/DOM runner (Layer B), Layer A runner.	**Complete.** The `perf-harness-wasm` crate, the Layer A runner, the orchestrator and the result schema's native environment. Repeatable figures: over five fresh-Chrome repetitions the construction medians of a 200-node build agree to within 5% (nodes 2%, edges 5%, total 3%). The 1,000-move run gives 40 batched observations a repetition, whose medians span 7.0 to 8.2 us (16%), so no p95 is reported. Correctness-mode checkpoints pass at 1,000 nodes and 3,000 edges. The counting allocator is verified with a deliberately retained allocation, a release and a leaked closure (section 5.2).
3	CDP runner (Layer C) and gallery adapter. *Conditional on a spike that proves the frame correlation of 3.2.*	Interaction and presentation figures, resource counts and the gallery sweep captured. Pan, wheel zoom and drag at three view scales; the wheel case is isolated, since a real wheel reaches only the active tab.
4	Recovery and endurance runners.	Each fault point recovers according to its documented contract, or is listed as a finding.
5	CI integration, baselines, comparison and history.	A deliberately injected regression and an injected listener leak are each caught by the harness's own tests; a clean fixture produces neither.

10.1 Testing the harness

- Native unit tests for percentiles, bootstrap intervals, trend slope, Tukey flags, the PRNG, the stream hash, the schema and the comparison verdicts, run with the per-commit tests.
- Known-answer tests: a seed's first values and stream hash are fixed in the repository.
- Every test returns `Result<(), String>`, following the crate's convention.

11 Decisions

Question	Decision
Crate layout	Two on-demand workspace members: `perf-harness` (manifests, workload generation, statistics, reports) and `perf-harness-wasm` (Layer B, consuming the shared workload definitions and the public API). Manifests in `perf/manifests/`. Neither adds a dependency to the published library.
Baselines	`perf/baselines/`, written only by an explicit promotion command.
`perf-smoke` in CI	From Phase 5. It fails on an incorrect checkpoint, a leaked deterministic resource or a malformed workload. It never fails on a statistically uncertain timing comparison.
Chrome-only memory	Accepted as diagnostic evidence. Counts and wasm memory are the cross-browser figures.
TOML	Accepted, confined to the harness tooling.

11.1 Still open

- Whether the frame correlation rule holds (Phase 3 spike).
- Whether the Phase 0 findings hold in Firefox and Safari. Nothing was run in either.

svg-dom-graph / performance harness / design specification rev 3. Derived from the 4 November 2026 coverage audit and two rounds of review. Thresholds are set from measurement, not from this document.